

Programme	B.Sc Computer Science				
Course Title	Compiler Design				
Type of Course	Major				
Semester	VIII				
Academic Level	400-499				
Course Details	Credit	Lecture per week	Tutorial per week	Practical per week	Total Hours
	4	4	-	-	60
Pre-requisites	1. Formal Languages & Automata Theory. 2. Data Structure and Algorithms				
Course Summary	This course covers the fundamental concepts of different phases of compilation such as lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization and code generation. Students can apply this knowledge in design and development of compilers.				

Course Outcomes (CO):

CO	CO Statement	Cognitive Level*	Knowledge Category#	Evaluation Tools used
CO1	To identify different phases in compilation process and model a lexical analyser.	Ap	P	Practical Assignment / Instructor-created exams / Quiz
CO2	To model language syntax using Context Free Grammar and develop parse tree representation using leftmost and rightmost derivations.	Ap	P	Practical Assignment / Instructor-created exams / Quiz
CO3	To compare different types of parsers and construct parser for a given grammar.	Ap	P	Practical Assignment / Instructor-created exams / Quiz
CO4	To build Syntax Directed Translation for a context free grammar, compare various storage allocation strategies and classify intermediate representations.	Ap	P	Practical Assignment / Instructor-created exams / Quiz
CO5	Students will demonstrate the ability to design and implement lexical analyzers to recognize tokens in source programs.	Ap	P	Practical Assignment / Instructor-created exams / Quiz

CO6	Illustrate code optimization and code generation techniques in compilation	Ap	P	Practical Assignment / Instructor-created exams / Quiz
* - Remember (R), Understand (U), Apply (Ap), Analyse (An), Evaluate (E), Create (C) # - Factual Knowledge(F) Conceptual Knowledge (C) Procedural Knowledge (P) Metacognitive Knowledge (M)				

Detailed Syllabus:

Module	Unit	Content	Hrs	Max Marks
I	COMPILERS AND LEXICAL ANALYSIS		10	15
	1	Analysis of the source program - Analysis and synthesis phases	2	
	2	Phases of a compiler, The grouping of Phases	2	
	3	Compiler writing tools. Bootstrapping.	2	
	4	Lexical Analysis : Parsing, Abstract stack machine, Role of Lexical Analyser	2	
	5	Input Buffering, Specification of Tokens, Recognition of Tokens.	2	
II	SYNTAX ANALYSIS		18	25
	6	Role of the Syntax Analyser, Role of the Parser	2	
	7	Context-free grammars, Parse Tree and Derivations, Eliminating Ambiguity	2	
	8	Basic Parsing Approaches – Eliminating left recursion, left factoring	3	
	9	Top Down parsing - Recursive Descent Parsing	2	
	10	Predictive Parsing - LL(1) Grammars	3	
	11	Bottom-up parsing -Handle Pruning - Shift Reduce Parsing - Operator Precedent Parsing	3	
	12	LR Parsers - SLR Parser- Canonical LR Parser - LALR Parser	3	
III	SEMANTIC ANALYSIS AND INTERMEDIATE CODE GENERATION		10	
	13	Syntax directed translation - Syntax directed definitions	2	15
	14	S-attributed definitions, L-attributed definitions, Bottom-up evaluation of S-attributed definitions. Run-Time Environments	2	
	15	Source Language issues, Storage organization, Storage-allocation strategies.	2	
	16	Intermediate Code Generation - Intermediate languages, Graphical representations,	2	
	17	Three-Address code, Quadruples, Triples.	2	
IV	CODE OPTIMIZATION AND CODE GENERATION		10	15
	18	Code Optimization - Principal sources of optimization	2	
	19	Machine dependent and machine independent optimizations,	2	

	20	Local and global optimizations.	2	
	21	Code generation - Issues in the design of a code generator,	2	
	22	Target Language, A simple code generator.	2	
V	Open Ended Module – Application Level		12	
		1. Learn the fundamentals of lexical analysis and parsing using Lex and Yacc, essential tools in compiler construction. 2. Apply the concepts learned to develop a small compiler, progressively enhancing its functionality while implementing error handling and optimization strategies. 3. Apply the concept of Bootstrapping and its significance in compiler construction.		
		4. Understanding of run-time environments and storage allocation strategies. 5. Development of a simple code generator for translating intermediate code into target code.		

Textbooks :

1. Aho A.V., Ravi Sethi and D. Ullman. Compilers – Principles Techniques and Tools, Addison Wesley, 2006.

Reference books :

1. D.M.Dhamdhere, System Programming and Operating Systems, Tata McGraw Hill &Company, 1996.
2. Kenneth C. Loudon, Compiler Construction – Principles and Practice, Cengage Learning Indian Edition, 2006.
3. Tremblay and Sorenson, The Theory and Practice of Compiler Writing, Tata McGraw Hill & Company, 1984.
4. Compiler Design in C, Allen I. Holub, Prentice Hall (Software Series).
6. Crafting a Compiler with C, C. N. Fischer and R. J. LeBlanc, Pearson Education.
7. Allen I Holub, Compiler Design in C, 1st Edition, PHI Learning Pvt Ltd.

Mapping of COs with PSOs and POs:

	PSO1	PSO2	PSO3	PSO4	PSO5	PSO6	PO1	PO2	PO3	PO4	PO5	PO6
CO 1	2	1	2	2	2	-						
CO 2	2	1	2	2	2	-						
CO 3	2	1	2	3	3	-						
CO 4	2	-	2	3	3	-						

CO 5	2	-	2	2	2	-						
CO 6	-	-	2	1	2	-						

Correlation Levels:

Level	Correlation
-	Nil
1	Slightly / Low
2	Moderate / Medium
3	Substantial / High

Assessment Rubrics:

- Quiz / Assignment/ Quiz/ Discussion / Seminar
- Midterm Exam
- Programming Assignments (20%)
- Final Exam (70%)

Mapping of COs to Assessment Rubrics:

	Internal Exam	Assignment	Practical Evaluation	End Semester Examinations
CO 1	✓	✓		✓
CO 2	✓	✓		✓
CO 3	✓	✓		✓
CO 4	✓	✓		✓
CO 5	✓	✓		✓
CO 6	✓	✓		✓